

FPGA-based Network Switch Architecture Supporting Credit Based Shaper for Time Sensitive Networks

Akram BEN AHMED, Takahiro HIROFUCHI, Takaaki FUKAI

Digital Architecture Research Center

National Institute of Advanced Industrial Sciences and Technology

2-3-26 Aomi, Koto-ku, Tokyo, JAPAN

{akram.benahmed,t.hirofuchi,takaaki.fukai}@aist.go.jp

Abstract—Time Sensitive Network (TSN) is one of the most auspicious solutions to respond to the increasing demands in ultra-low latency in the 5G and post-5G eras. It comes as an extension to the conventional IEEE 802.3 Ethernet networks by adding a set of novel open standards that aims to provide deterministic, reliable, high-bandwidth, and low-latency communication. In this paper, we present an open-source and light-weight FPGA-based network switch design and implementation supporting Credit Based Shaper (CBS) for Time Sensitive Networks. We present the key design components and implementation aspects of the proposed switch and discuss the preliminary evaluation results in a fair amount of detail to validate our proposal. The conducted experiments show that the proposed switch properly shapes the traffic by eliminating bursts in irregular traffic and efficiently forwards prioritized traffic as stipulated by the TSN requirements. In addition, we demonstrate that our hardware latency evaluation results conform with the CBS theoretical model and that the proposed switch consumes a very reasonable portion of the hardware resources on an affordable FPGA.

Index Terms—Time Sensitive Networks, Credit Based Shaper, FPGA, Switch Architecture

I. INTRODUCTION

The rapid advancement of technology has led to a significant increase in the connectivity requirements within networks. This growth is particularly notable with the emergence of new technological paradigms such as the Internet of Things, Edge computing, and 5G communication. Consequently, there has been an exponential rise in the volume of data exchanged and shared. As new functionalities are regularly introduced, the demands on communication infrastructure continue to escalate. This trajectory is projected to persist, especially with the growing reliance on critical applications like autonomous driving, cyber-physical systems, and industrial automation.

Within these applications, the importance of time-sensitive traffic with ultra-low latency (ULL) of milliseconds or less has become paramount, as noted by previous studies [1], [2]. Any latency that fails to meet expectations could result in degraded performance, potential risks, or even catastrophic consequences in terms of material damage or human safety. Hence, there is an urgent need for robust communication standards capable of guaranteeing bounded low latency, minimal

delay variation, and extremely low data loss for time- and mission-critical traffic.

In response to these demands, Time Sensitive Networks (TSN) have seen widespread adoption [3]. TSN, developed by the IEEE 802.1 Time Sensitive Networking (TSN) Task Group, introduces additional standards to conventional IEEE 802.3 Ethernet networks aimed at providing deterministic guarantees, robustness, and integrated diagnostics and management services [4]. Notably, some of these standards are gradually being integrated into the primary Ethernet Bridge standard (IEEE 802.1Q-2022) as they become default requirements.

Several industrial TSN switch prototypes and solutions have been developed. However, architecture or implementation details are not publicly available. Most of the already conducted academic studies have focused on characterizing these algorithms theoretically or through simulation under different traffic types. On the other hand, only a few academic works have tried to implement and evaluate TSN functionalities on hardware (ASIC or FPGA) rather than simulation-based solutions and analysis. The need for openly available reconfigurable FPGA-based TSN designs has become important to improve TSN research.

In this paper, we propose an open-source hardware design and implementation of an FPGA-based switch towards TSN support [5]. The proposed architecture hosts multiple queues for different traffic classes and we designed a fixed priority arbitration mechanism between these traffic classes. For each traffic class, we assigned a transmission selection mechanism. As a first step, we choose the Credit Based Shaper (CBS) scheduling algorithm as one of the traffic selection mechanisms. Apart from a conventional network switch, we carefully design the data path of frames and properly place the computation and memory resources to achieve latency bound based on the TSN standard model.

Our design is highly customizable and lightweight. We designed it in a way to be portable to most affordable mid-size and low-power FPGA boards, widely available in the market, which will be usable for developing network switches and end-points in edge-computing and other timing-critical applications. We aspire that researchers will adopt, reuse, and

optimize our design to their research platforms and requirements. Moreover, CBS has been widely studied and can be a perfect candidate for preliminary evaluation. Through our evaluation experiments, we successfully verify the switch's capabilities in terms of traffic shaping, prioritization, and adherence to upper-bound latency, as stipulated by the TSN standards.

The structure of this paper is outlined as follows: Section II provides a succinct overview of the key characteristics and operation of CBS. Section III discusses some of the related works to TSN evaluation and implementation. In Section IV, we present our FPGA-based switch design and implementation supporting CBS. Section V is dedicated to the preliminary evaluation, validation, and analysis of our proposed implementation. We discuss the potential practical use of our implementation in Section VI before we conclude the paper with summarizing remarks in Section VII.

II. CREDIT BASED SHAPER OVERVIEW

Credit Based Shaper (CBS) is a scheduling algorithm which is one of the fundamental features of TSN as it facilitates the transportation of both critical and non-critical traffic across a bridged network with shared resources, accommodating various Quality-of-Service (QoS) requirements to ensure latency bounds [3]. Before explaining how CBS works, it is important first to understand the key components of a given TSN architecture.

A. Typical TSN Composition

In a typical TSN network, the infrastructure comprises multiple bridges and end-stations, where an end-station can function as a talker (sender), a listener (receiver), or both. Data flows originating from a talker are termed TSN streams, traversing the network via bridges (switches). End-stations and their associated streams may have distinct Quality-of-Service (QoS) requirements, which are conveyed to bridges for resource reservation via Centralized User Configuration (CUC) and Centralized Network Configuration (CNC) (previously referred to as IEEE 802.1Qcc), or optionally through the Stream Reservation Protocol (previously referred to as IEEE 802.1Qat). Furthermore, each egress port in a bridge can accommodate up to eight output queues (referred to as Traffic Classes), each assigned to a specific priority level.

In addition to traffic shaping, the IEEE 802.1Qav standard, later merged to primary the Ethernet Bridge standard (IEEE 802.1Q-2022) [3], mandates the inclusion of an arbiter responsible for ensuring the proper forwarding of packets based on their traffic class priority. As previously discussed, traffic is divided into eight classes, with each class assigned to one of the eight egress queues depending on the priority indicated in the Priority Code Point (PCP) field within the VLAN tag of the frame. Consequently, queue priorities are strictly adhered to ensure that frames are selected from higher priority queues and forwarded over lower priority or best-effort traffic streams.

To ensure adherence to QoS requirements, TSN networks support various traffic shaping algorithms (also known as

scheduling or queuing mechanisms). These algorithms include Strict Priority (SP), Asynchronous Traffic Shaper (ATS), and Credit Based Shaper (CBS), among others [3]. Each algorithm exhibits unique characteristics tailored to different QoS demands based on the deployed application. As previously indicated, we opted in this paper for CBS, and we proceed to outline its primary features and principles.

B. CBS Traffic Shaping

CBS aims at regulating the transmission rate of Ethernet frames and minimizing traffic fluctuations and bursts. This can be achieved by restricting the bandwidth that a particular traffic class can utilize and intentionally introducing buffering delays to specific packets.

In CBS, each of the eight egress queues is associated with a credit counter. When a queue is empty, the credit counter accumulates. As frames are transmitted, the credit counter is depleted. CBS manages this credit counter through parameters known as *idleSlope* and *sendSlope*, which regulate the rate of credit accumulation and consumption. For the sake of clarity, we assume a scenario where a packet arrives at a queue while another conflicting packet is being transmitted. Upon completion of the conflicting packet's transmission, the credit queue counter is inspected. If the counter is zero or positive, the packet is deemed ready for transmission from the perspective of the queue's exit side, enabling the priority control multiplexer to dequeue the packet. Otherwise, the frame remains in the queue and experiences delay until the credit counter once again becomes zero or positive.

III. RELATED WORK

In this section, we highlight some of the works conducted so far that are tackling challenges TSN. As we will see, only a few academic works have proposed the design of TSN functionalities which makes the need for openly available reconfigurable FPGA-based designs important to improve TSN research.

Numerous works have undertaken surveys to explore TSN technologies and their applications in recent years. For example, *Seol et al.* [6] conducted a comprehensive general survey, categorizing and discussing various TSN technologies and methodologies. Additionally, surveys have focused on the application of TSN in automotive and intelligent driving [7], as well as in smart factories [8].

Apart from surveys, various studies have focused on simulating different behaviors of TSN networks and the traffic they convey. For instance, *Lobello et al.* [9] tried to simulate and analyze the effects of various types of traffic classes on several traffic scheduling and shaping algorithms. Similarly, works by *Jiang et al.* [10] and *Falk et al.* [11] proposed simulation models for TSN using *OMNET++* to explore different TSN features. It is noteworthy that many earlier studies from the AVB (Audio Video Bridging) domain transitioned to TSN [12], [13]. Although simulation-based approaches offer valuable insights into TSN behavior, they may suffer from accuracy limitations inherent to software simulations.

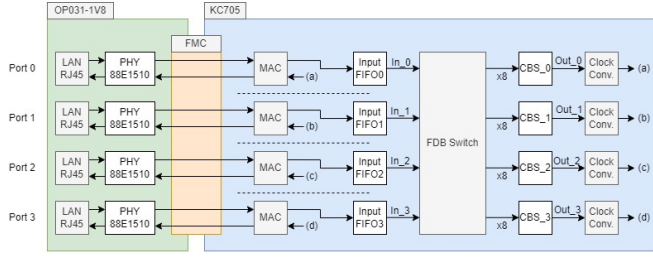


Fig. 1. Proposed FPGA-based CBS-switch

In the realm of hardware-based solutions, numerous industrial TSN switches and solutions have been developed. However, detailed implementation specifics are not openly provided. Conversely, academia has seen relatively few studies focusing on TSN hardware methodologies. *Zhou et al.* [14] proposed an isolated implementation of an ATS scheduling entity without developing the entire switch.

Quan et al. [15] is probably the only work that also aspires to provide an open-source framework for TSN development. However, the Github repository that they are providing does not specify any license. This makes reusing, modifying, and sharing their project problematic. Moreover, the materials provided are mostly not written in English. These factors make the authors' open-source claims, unfortunately, obsolete.

IV. PROPOSED FPGA-BASED TSN SWITCH HARDWARE DESIGN AND IMPLEMENTATION

The proposed FPGA-based switch is designed to support CBS capabilities and 1000BASE-T with a maximum speed of 1 GbE. It accommodates two Ethernet frame formats: Ethernet II and IEEE 802.3ac (VLAN tag support). Furthermore, the Maximum Transmission Unit (MTU) is set to 1500 bytes.

As depicted in Figure 1, the primary components of our switch are implemented on an AMD Xilinx Kintex 7 FPGA KC705 Evaluation Kit [16]. Since the KC705 board features only a single Ethernet port, we have augmented it with an *Opsero OP031-1V8 Ethernet FMC* [17], connected via the “FMC HPC” connector, as shown in Figure 1. This expansion card hosts four *Marvell 88E151x* Gigabit Ethernet PHYs, providing four ports of gigabit Ethernet connectivity to the KC705 board.

As stated earlier, it is important to mention that our design is not limited to the current implementation on KC705. As the majority of our design is written in Verilog-HDL with minimum deployment of Xilinx IP cores, We expect that only minor modifications are necessary to port our design to other FPGA boards.

A. MAC block

Incoming streams from a specific Ethernet port are initially directed to the MAC block, which facilitates data bridging between the PHY (utilizing the RGMII interface) and the FPGA programmable logic section via an AXI4-Stream bus. Within the MAC block, incoming streams undergo processing where the Preamble, Start Frame Delimiter (SFD), and

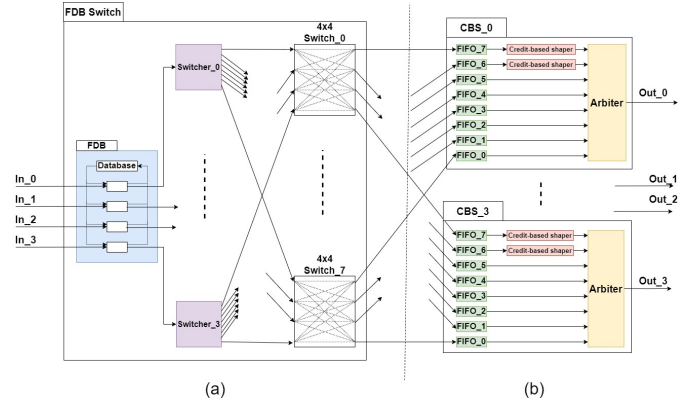


Fig. 2. Simplified block diagram of the interconnection between (a) FDB Switch and (b) CBS block

Physical Coding Sublayer (PCS) fields are removed from the input Ethernet frames. Conversely, these fields are added for outgoing frames. The MAC block comprises the Tri-Mode Ethernet Media Access Controller (TEMAC) module, which is an official soft IP from AMD/Xilinx for the Ethernet MAC layer [18] (not embedded in the PS section), alongside the “eth_ctrl”, a modified sample code of the TEMAC IP designed to support four-channel functionality. It is worth mentioning that as of writing, the AMD provides 1G TEMAC IP for free of charge for 120 days, and its project license without that restriction can be purchased, if necessary.

B. Pre-switch FIFO block

Frames are subsequently stored in the input FIFO blocks, as depicted in Figure 1, before being directed to the switch. These FIFO blocks serve for both storage and clock conversion functions, facilitating synchronization between the MAC side and the FPGA core side. Although both domains operate at the same clock frequency of 125MHz, they are driven from different clock sources, necessitating synchronization. The FIFO blocks are configured in “Packet Mode”, which delays the initiation of packet transmission until the end (LAST beat) of the packet is received. Consequently, latency increases proportionally with the length of the stream (equivalent to the frame length), enhancing transmission reliability.

C. FDB Switch block

The “FDB Switch” block functions as an L2 Switch with Filtering Database (FDB) capability. The FDB is responsible for storing the MAC address information for each port and directing Ethernet frames only to specific ports based on this information. Our design features four input ports, each corresponding to one of the four physical Ethernet input ports. For outputs, the “FDB Switch” has a total of 32 outputs, with each set of eight outputs (coming from each of the eight “4x4_Switch”) connected to one of the four CBS blocks representing the four physical Ethernet output ports, as represented in Figure 2 (a). This division of channels into eight links for each CBS block accommodates the eight priorities

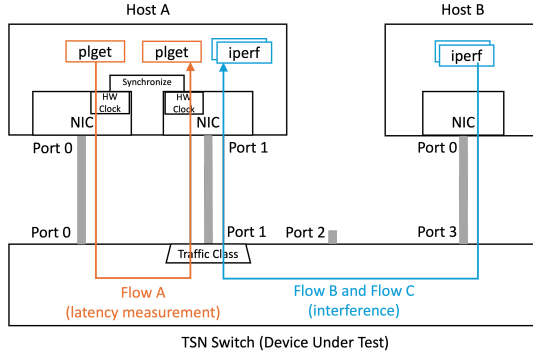


Fig. 3. Experiments setup for latency measurement

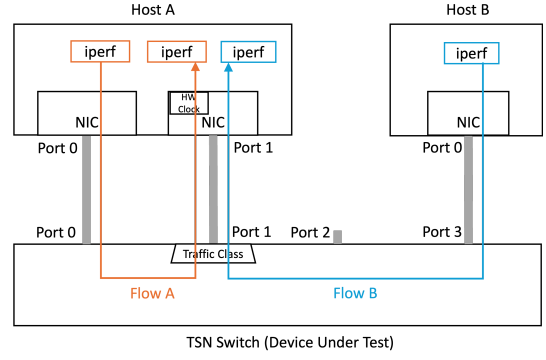


Fig. 4. Experiments setup for rate measurement

associated with each physical port, as dictated by the PCP field in accordance with the TSN standard, as described in Section II-A.

D. CBS block

This block manages the CBS functionalities within the proposed switch, giving priority to frames with high priority and outputting them accordingly. However, switching is not performed during frame output but earlier in the “FDB Switch” block. Subsequently, once a frame has been output, the next frame with the highest priority is selected for output. As depicted in Figure 2 (b), the CBS block features an input FIFO buffer for each of the eight input streams incoming from the “FDB Switch,” as described earlier in Section IV-C. Subsequently, if the traffic class has a priority of either “7” or “6,” the streams are directed to the “Credit-based Shaper” module, where the CBS *idleSlope* and *sendSlope* parameters are managed. Upon exiting the CBS block, the streams undergo clock conversion before being routed back to the MAC blocks.

V. EVALUATION

In this section, we evaluate our TSN switch prototype. To simplify the discussion, otherwise noted, the payload size of all the frames is set to 1500 bytes, which means the size of an Ethernet frame encoded at the physical layer is 1542 bytes including Inter-Frame Gap (12 bytes), Preamble (8 bytes), the Ethernet header of the VLAN support (18 bytes), and Frame Check Sequence (4 bytes).

A. Measurement setup

Figures 3 and 4 illustrate the experiment setups. Host A has two GbE NICs (Intel i210) and Host B has one GbE NIC. They are connected to the ports of the TSN switch, respectively. The TSN switch forwards frames from Port 0 and Port 3 to the traffic classes of Port 1. It is expected to ensure the worst-case delays of priority flows according to the configuration of the traffic classes.

Intel i210 NIC has an internal hardware clock mechanism that is intended to maintain TAI (International Atomic Time) for the support of Precision Time Protocol (PTP) [19]. It can

accurately record the transmission and arrival times of frames at the hardware level. Thanks to this mechanism, we measured the arrival intervals of frames in Section V-C and the end-to-end latency of frames in Sections V-B, V-E, and V-F in an 8 ns resolution.

To measure the end-to-end latency of a frame, it is necessary to record its transmission time at the source and record its arrival time at the destination. Both source and destination NICs need to synchronize their internal hardware clock with each other. We avoid using the PTP protocol for clock synchronization because PTP messages via the network switch are likely to disturb the evaluation of the network switch itself. Instead, we use one machine equipped with two Intel i210 NICs and periodically synchronize their hardware clocks.

As a preliminary experiment, we first investigate the accuracy of the measured latency in the used setup and confirmed that it is safe to discuss it in a 1 us resolution. We connected Port 0 and Port 1 of Host A with an Ethernet cable and measured the network latency between them. Since there is only a fixed-length cable between them, the network latency is expected to be constant. Although configurations were carefully tuned up, the measured latency varied from -500 ns to +500 ns. The program to synchronize the hardware clocks (i.e., *phc2sys*, a supplementary program in the Linux PTP package) is executed on the operating system. It reads the PTP Hardware Clock (PHC) from the Intel i210 chip of a NIC and writes it to the other. Since this synchronization is done at the software layer, it is inevitably affected by any irregular behavior of the operating system (e.g., task scheduling and interrupt handling).

Second, we examined the safe frame rate for latency measurement. Since the hardware timestamp is designed for sporadic PTP messages, Intel i210 NIC can record the transmission hardware timestamp only for one frame in the hardware transmission queue. If multiple frames exist in the queue, correct transmission times are not obtained. We carefully choose the frame rate of a flow for the latency measurement and decided to use 100 frames per second (i.e., 1.2 Mbps with 1542B frames) for the remaining experiments.

We used a latency measurement program, *plget*, with our minor customization. It sends/receives frames while reading

TABLE I
BASIC LATENCY OF TRANSMISSION SELECTION ALGORITHMS

Header + Payload (bytes)	Transmission Selection Algorithm		
	CBS 100Mbps (us)	CBS 10Mbps (us)	Strict Priority (us)
14 + 64	4	4	4
14 + 1500	50	50	50

their hardware transmit/receive timestamps via the socket API of the Linux Kernel. These timestamps are recorded at the hardware level with a granularity of 8 ns. We also employed a frame transmit/receive program, *iperf*, to generate frames at a line rate of 1 GbE. To record frame intervals, we chose a packet capture program, *tcpdump*, with the hardware timestamp option.

The used machines are equipped with an Intel i7-7700 CPU, which is sufficiently powerful to generate frames at the line rate of 1 GbE. To mitigate the fluctuation of the measured latency, we fixed the frequency of the CPU cores at their maximum speed and also disabled their temporal sleep mechanism (i.e., Intel C-state).

B. Network switch basic latency

First, we measure the basic latency of the implementation of each transmission selection algorithm. We generate only the flow of frames for latency measurement by *plget* and did not generate any other interference flows, as shown in Figure 3 (Flow B and C are not considered here). For CBS, *idleSlope* was set to 100 Mbps and then to 10 Mbps. Since the transmission rate of *plget* was set to 1.2 Mbps, which is sufficiently smaller than the set *idleSlope* values, no frames were delayed by the rate control of CBS. The basic latency of the switch to forward a frame without any queuing by rate control or contention is measured. Strict Priority is also used for comparison.

Table I summarizes the obtained results. As long as we have a 1 us resolution of the measurement setup, no difference among the transmission selection algorithms was observed. The latency is 50 us for 1500B frames and 4 us for 64B frames. As shown in Figure 5, the latency is proportionally related to the frame size. As discussed later, because the CBS module consumes 8 cycles for a frame, the latency of the frame passing it should be 64 ns longer than that of the Strict Priority.

We confirmed that these obtained latency values are consistent with the cycles consumed in the developed FPGA components. The theoretical model in Figure 5 (red line) is based on the below discussion and matches the obtained results. We used Integrated Logic Analyzer (ILA) to see how much each FPGA component of the TSN switch contributes to the overall latency. The developed components consume the amount of cycles shown in Table II. As the frame size increases, the cycles consumed in the “Pre-switch FIFO” and “FDB Switch” increase approximately by the frame size, respectively. The cycles consumed in “eth_ctrl”, previously explained in Section IV-A, increase approximately by 2 times

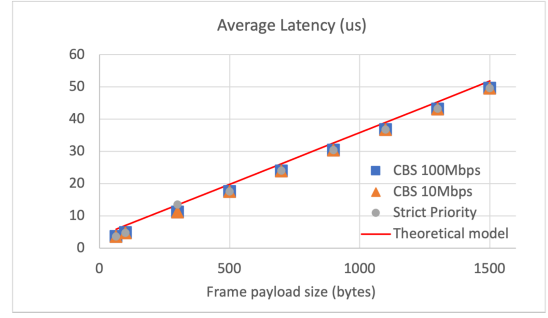


Fig. 5. Basic latency for different payload sizes

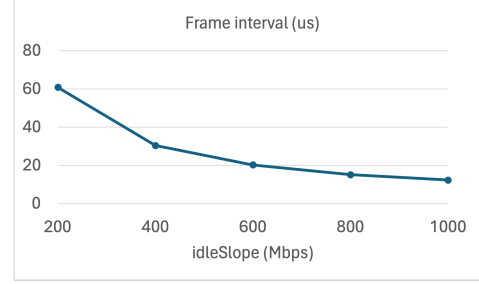


Fig. 6. Measured frame intervals with different shaping rates

the frame size. According to the datasheet of the used PHY transceiver (Marvell 88E151x) [20], the receive latency is between 227 ns to 235 ns, and the transmit latency is between 141 ns to 153 ns. The receive latency of the used Ethernet MAC IP (Xilinx Tri-Mode Ethernet MAC) [18] is 15 cycles (i.e., 120 ns) and its transmit path latency is 8 cycles (i.e., 64 ns). The obtained latency value of PHY + TEMAC, 562 ns, is explained by these values in the datasheets. In summary, the analysis by ILA showed that the total latency for a frame varies from 4 us to 50 us, depending on the frame size.

C. Rate control by CBS

We evaluate the rate control capability of the developed CBS transmission selection module. Figure 4 depicts the setup of this experiment (Flow B is not considered here). We assign CBS to Traffic Class 7 at the TSN switch and change its shaping rate through the *idleSlope* parameter from 200 Mbps to 1000 Mbps. At Host A, *iperf* generates UDP frames for Traffic Class 7 at the maximum frame rate of 1 GbE with a 1542-bytes physical frame size. At the receiving port of Host A, we measure the arrival times of 1001 frames in a row by using *tcpdump* with the hardware timestamp option.

Figure 6 and Figure 7 show the average of 1000 frame intervals in microseconds (us) with different *idleSlope* values and their estimated arrival rates in Mbps, respectively. Overall, our CBS module successfully extends the frame intervals to the ones required by the *idleSlope* values. For example, the theoretical frame interval for 200 Mbps is 61.680 us. The measured intervals were 60.880 us for 96.2% of the frames and 60.872 us for 3.7%. In this paper, we consider the necessity to validate the measurement mechanism itself. However, we will

TABLE II
LATENCY IN EACH FPGA COMPONENT. 1 CYCLE CORRESPONDS TO 8 NS.

Header + Payload (bytes)	Component					
	Eth driver (cycles)	Pre-switch FIFO (cycles)	FDB Switch (cycles)	CBS (cycles)	PHY + TEMAC (ns)	Total (ns)
14 + 64	176	91	108	5	562	4338
14 + 1500	3048	1527	1544	5	562	61778

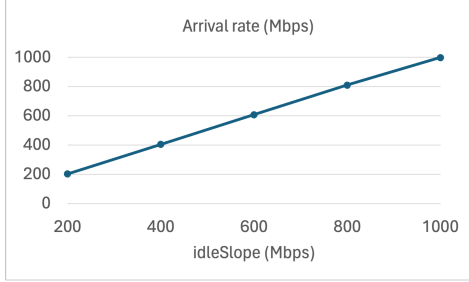


Fig. 7. Arrival rates calculated from the measured frame intervals

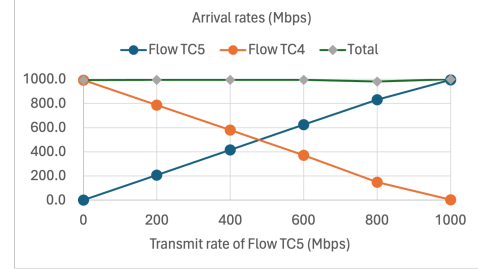


Fig. 8. Arrival rates of frames of Traffic Class 4 and 5

investigate this minor difference from the theoretical interval in the future. Considering this is the first prototype, the TSN switch demonstrated the promising capability of accurate rate control.

Note that the TSN switch drops excessive frames when the input rate of a flow continuously exceeds its *idleSlope*. In our design, the queue size of CBS is configurable as long as a target FPGA chip can accommodate. In real use cases, a sender host transmits frames within the *idleSlope* rate agreed with the TSN switches along the path of the flow. The CBS schedulers precisely adjust the frame intervals, to assure that frames are not buffered unexpectedly for a long time at any hop. As long as a sender host transmits frames at the agreed rate, CBS assures that no frame is dropped and the latency is bounded.

D. Fixed priority arbitration among traffic classes

A TSN switch needs to implement a fixed priority arbiter among traffic classes. The combination of the rate control of a traffic class and its fixed precedence to other traffic enables the modeling of the worst-case delay for a flow passing through the traffic class. A traffic class of a higher index number must have absolute priority over a traffic class of a lower index number. If and only if the highest traffic class (i.e., Traffic Class 7) does not have any frames that are ready for transmission, the next highest traffic class has a chance to transmit its frames. This is repeated down to the lowest traffic class.

To confirm that the fixed priority arbiter gives strict priority to a higher traffic class, we generated two flows of frames that are merged at the switch, as illustrated in Figure 4. *iperf* on Host A generates frames for Traffic Class 5, and that of Host B generates frames for Traffic Class 4. Both flows are merged at Port 1 of the switch. To simplify the discussion, we choose the traffic classes of the Strict Priority schedulers.

Although not discussing its details, we confirm that the traffic classes of the CBS scheduler also correctly work with the fixed priority arbiter. The fixed priority arbiter is independent of the transmission selection mechanisms of the traffic classes.

While *iperf* on Host B always generates frames for Traffic Class 4 at the line rate of 1 GbE, that of Host A generates frames for Traffic Class 5 at different rates. Figure 8 shows the arrival rates of frames of Traffic Class 5 and 4, respectively. Despite the existence of another flow, the arrival rate of the Traffic Class 5 flow was the same as its transmission rate, and no frame drops were observed. In contrast, the arrival rate of the Traffic Class 4 flow was severely suppressed by the higher priority flow. The number of observed frame drops increased correspondingly. Evidently, the fixed priority arbiter worked as intended.

E. Latency bound of the highest traffic class

The theoretical model allows us to estimate the worst-case delay of frames waiting in the queue of a CBS traffic class. Even though the frame at the head of the highest traffic class is ready for transmission, it needs to wait for the completion of the ongoing transmission of a frame from a lower traffic class. As shown in Equation L-39 in [3], the maximum of this queuing delay in Traffic Class 7 is calculated as:

$$qDelay_A = M_0/R_0 = 1542B/1Gbps = 12.336\mu s$$

where M_0 is the maximum size of frames transmitted from the port, and R_0 is the port transmission rate in the experiments. Figure 9 illustrates the credit of Traffic Class 7 and the timing of frame transmission. In this case, just after a frame of Traffic Class 5 starts being transmitted, the tail bit of a frame of Traffic Class 7 arrives. Even though Traffic Class 7 has precedence, it needs to wait for 12.336 μs until Traffic Class 5 finishes transmitting the ongoing frame.

To confirm that the actual worst-case delay matches the theoretical value, we set up experiments, as illustrated in

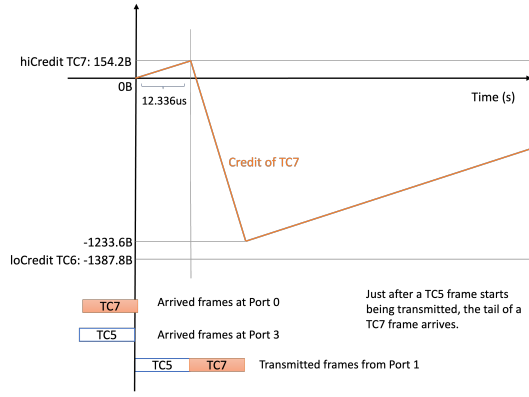


Fig. 9. The credit of Traffic Class 7 and the timing of frame transmits

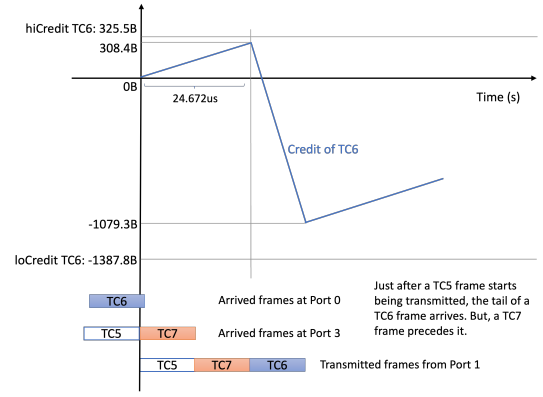


Fig. 10. The credit of Traffic Class 6 and the timing of frame transmits

Figure 3. The *idleSlope* of the CBS scheduler of Traffic Class 7 was set to 100 Mbps and the transmission selection of Traffic Class 5 was set to Strict Priority. `plget` on Host A generates a flow of frames for latency measurement at 1 Mbps. `iperf` on Host B generates an interference flow of frames at 900 Mbps or 100 Mbps. The maximum observed latency was approximately 74 us, which is a 12-us increase from the latency without the interference flow (i.e., 62 us in Section V-B). The average latency with the 1 Gbps interference flow was 69 us, and that with the 100-Mbps interference flow was 62 us. As the rate of the interference flow is higher, frames of the higher traffic class are more likely to be delayed due to contention. The obtained worst-case delay fairly matches the theoretical one.

F. Latency bound of the second highest traffic class

The theoretical model also enables us to estimate the worst-case delay of the second highest traffic class with CBS assignment. According to Equation L-41 in [3], it is calculated as:

$$qDelay_B = (M_0 + M_A) / (R_0 - R_A)$$

$$= (1542B + 1542B) / (1Gbps - 100Mbps) = 27.413us$$

where M_A is the maximum frame size in Traffic Class 7, and R_A is its *idleSlope*.

In addition to the setup for the experiment presented in Section V-E, the *idleSlope* of Traffic Class 6 was set to 100 Mbps. To measure the latency of a flow of Traffic Class 6, `plget` on Host A generates a flow of frames for Traffic Class 6. One instance of `iperf` on Host B generates a competing flow at 100 Mbps for Traffic Class 7, and the other `iperf` instance generates the other interference flow at 900 Mbps for Traffic Class 5, corresponding to Flow B and C in Figure 3, respectively.

The maximum contention delay observed in the flow of Traffic Class 6 was 25 us, which was under the theoretical value. We consider that the worst-case delay in Traffic Class 6 happens in the situation illustrated in Figure 10. Just after a frame of Traffic Class 5 started its transmission, the tail of a frame of Traffic Class 6 arrives and now Traffic Class 6

has the ready frame. However, when the transmission of the Traffic Class 5 frame finishes, Traffic Class 7 also has a ready frame, and the transmission of the frame of Traffic Class 6 is further postponed. In this case, the worst-case delay in Traffic Class 6 should be 24.672 us (i.e., a duration of time to transmit the 2 pending frames), which is equal to the measured value obtained in a 1-us precision.

G. Hardware resource utilization

We conclude our evaluation by presenting the hardware complexity of the proposed FPGA-based switch. As previously noted, our switch is implemented on an AMD Xilinx Kintex 7 FPGA KC705 Evaluation Kit [16]. Therefore, the resource utilization results were obtained using the AMD Xilinx Vivado 2022.1 tool [21]. Table III showcases the resource utilization of each component of our switch as a percentage of the available resources.

As depicted in this table, a significant portion of the Logic Lookup Tables (LUTs) is utilized by the “FDB Switch” as it manages the necessary multiplexing. Conversely, Memory Lookup Tables (LUTRAMs) and BRAM blocks are primarily consumed by the “Pre-switch FIFOs” and “CBS” blocks. This distribution is natural since these blocks include buffers for storing processed frames before forwarding them downstream.

Furthermore, it can be seen from this table that the proposed switch is lightweight. In fact, only 22% of the LUTs and less than 40% of the available BRAM blocks are utilized. It is worth noting that the KC705 evaluation board is among the most affordable and low-power FPGA development boards, featuring relatively limited resources compared to other mid- or top-tier commercially available counterparts. We assert that the current architecture enables the implementation of additional TSN features in the future while maintaining FPGA costs at a minimum. Furthermore, since the resource utilization of the proposed switch is fairly reasonable, users can implement this design on other cheaper FPGAs that do not require some of the peripherals found in KC705.

VI. TOWARDS PRACTICAL USE

In this paper, we made the necessary design steps for our FPGA-based switch to satisfy communication determinism

TABLE III
PROPOSED SWITCH HARDWARE COMPLEXITY

Resources / Component	MAC	Pre-switch FIFO	FDB Switch	CBS	Clock Converter	Others	Total	Utilization (%)
LUTs	9838	5536	19281	5977	266	4453	45351	22.25
FFs	15379	3716	16807	8868	448	5413	50631	12.42
LUTRAMs	1504	4096	32	4	32	230	5898	9.22
BRAMs	8	0	2	160	0	3	173	38.88

in the network and validated this feature through several experiments. Determinism is a fundamental criterion for TSN networks which ensure that a frame is transmitted from one point to another within a predetermined latency. On the other hand, we have observed an increase in latency. In fact, we showed in Table II that the switch latency increases proportionally with the frame size. This latency increase is in the us range which is still quite low and acceptable. However, for some users, this can be a concern in extreme use-case scenarios.

As previously explained, we opted for the “Packet Mode” configuration in many instances of the implemented buffers which delays the initiation of packet transmission until the end of the packet is received. This design choice aims to ensure deterministic communication and simple testing of the proposed switch. However, it adds a substantial amount to the overall latency, which is still coherent with TSN concepts. Nonetheless, this “Packet Mode” configuration can be omitted when the latency requirements are much stricter in other mission-critical applications that researchers may implement.

VII. CONCLUSION

In this paper, we proposed a hardware implementation of Credit Based Shaper functionalities on an FPGA-based switch for Time Sensitive Networks. We gave an overview of the proposed switch design and implementation on an affordable FPGA board and we validated its functionalities through several evaluations.

Our evaluations show that the proposed switch properly shapes the traffic by eliminating irregular bursts and efficiently forwarding streams depending on their priority, according to the TSN standards rules. Finally, we demonstrate that our hardware latency evaluation results conform with the CBS theoretical model. We provide a transparent, accessible, and open-source repository with software and hardware implementation details, and simple demos to demonstrate the implemented TSN capabilities. We aspire to allow researchers to reuse, modify, and optimize the current implementation by adding other TSN features.

ACKNOWLEDGMENT

This paper is based on results obtained from the project, “Research and Development Project of the Enhanced infrastructures for Post-5G Information and Communication Systems” (JPNP20017), commissioned by the New Energy and Industrial Technology Development Organization (NEDO).

REFERENCES

- [1] M. A. Al-Maqri, M. A. Alrshah, and M. Othman, “Review on qos provisioning approaches for supporting video traffic in ieee802.11e: Challenges and issues,” *IEEE Access*, vol. 6, p. 55202–55219, 2018.
- [2] M. Raza, H. Le-Minh, N. Aslam, S. Hussain, M. A. Imran, R. Tafazolli, and H. X. Nguyen, “Dynamic priority based reliable real-time communications for infrastructure-less networks,” *IEEE Access*, vol. 6, pp. 67 338–67 359, 2018.
- [3] *IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks*, Institute of Electrical and Electronics Engineers, 2022.
- [4] A. Nasrallah, A. Thyagaturu, Z. Alharbi, C. Wang, X. Shao, M. Reisslein, and H. Elbakoury, “Performance comparison of ieee 802.1 tsn time aware shaper (tas) and asynchronous traffic shaper (ats),” *IEEE Access*, vol. 7, pp. 44 165–44 181, 2019.
- [5] <https://github.com/CCIRT/aist-tsn-cbs>, 2024.
- [6] Y. Seol, D. Hyeon, J. Min, M. Kim, and J. Paek, “Timely survey of time-sensitive networking: Past and future directions,” *IEEE Access*, vol. 9, pp. 142 506–142 527, 2021.
- [7] Y. Xu and J. Huang, “A survey on time-sensitive networking standards and applications for intelligent driving,” *Processes*, vol. 11, no. 7, 2023.
- [8] T. Fedullo, A. Morato, F. Tramarin, L. Rovati, and S. Vitturi, “A comprehensive review on time sensitive networks with a special focus on its applicability to industrial smart and distributed measurement systems,” *Sensors*, vol. 22, no. 4, 2022.
- [9] L. Lo Bello, M. Ashjaei, G. Patti, and M. Behnam, “Schedulability analysis of time-sensitive networks with scheduled traffic and preemption support,” *Journal of Parallel and Distributed Computing*, vol. 144, pp. 153–171, 2020.
- [10] J. Jiang, Y. Li, S. H. Hong, A. Xu, and K. Wang, “A time-sensitive networking (tsn) simulation model based on omnet++,” in *2018 IEEE International Conference on Mechatronics and Automation (ICMA)*, 2018, pp. 643–648.
- [11] J. Falk, D. Hellmanns, B. Carabelli, N. Nayak, F. Dürr, S. Kehrler, and K. Rothermel, “Nesting: Simulating ieee time-sensitive networking (tsn) in omnet++,” in *2019 International Conference on Networked Systems (NetSys)*, 2019, pp. 1–8.
- [12] B. Fang, Q. Li, H. Hu, and H. Xiong, “Token regulated credit based shaper for time sensitive networks,” in *2020 International Conference on Computer, Information and Telecommunication Systems (CITS)*, 2020, pp. 1–6.
- [13] E. Mohammadpour, E. Stai, and J.-Y. Le Boudec, “Improved credit bounds for the credit-based shaper in time-sensitive networking,” *IEEE Networking Letters*, vol. 1, no. 3, pp. 136–139, 2019.
- [14] Z. Zhou, M. S. Berger, and Y. Yan, “Mapping tsn traffic scheduling and shaping to fpga-based architecture,” *IEEE Access*, vol. 8, pp. 221 503–221 512, 2020.
- [15] J. Y. Wei Quan, Wenwen Fu and Z. Sun, “Opentsn: an open-source project for time-sensitive networking system development,” *CCF Transactions on Networking*, vol. 3, pp. 51–65, 2020.
- [16] AMD Xilinx Cooperation, *KC705 Evaluation Board for the Kintex-7 FPGA, UG810 (v1.9)*, 2019.
- [17] O. E. D. Inc., <https://ethernetfmc.com/>, 2022.
- [18] Advanced Micro Devices, Inc., *Tri-Mode Ethernet MAC LogiCORE IP Product Guide (PG051)*, v9.0, 2023.
- [19] Intel Cooperation, *Intel Ethernet Controller I210 Datasheet, Revision Number: 3.7*, 2021.
- [20] Marvell Technology, *Marvell Alaska 88E1510/88E1518/88E1512/88E1514, Integrated 10/100/1000 Mbps Energy Efficient Ethernet Transceiver*, 2022.
- [21] AMD Xilinx Cooperation, *Vivado Design Suite User Guide: Getting Started (UG910)*, 2022.